IMEN574 / IDSC574 · PROGRAMMING FOR AX

21

Generative Models

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 Programming with AI

Lectures 2–4

02 Python and Classical Machine Learning

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: Generative Models

01 **Sampling Distributions**

From one answer to many

02 **Denoising Models**

Learn a recovery field

03 **Flow and Editing**

Control a generation path

01 · Sampling Distributions

01 **Sampling Distributions**

From one answer to many

02 **Denoising Models**

Learn a recovery field

03 **Flow and Editing**

Control a generation path

Why Model a Distribution

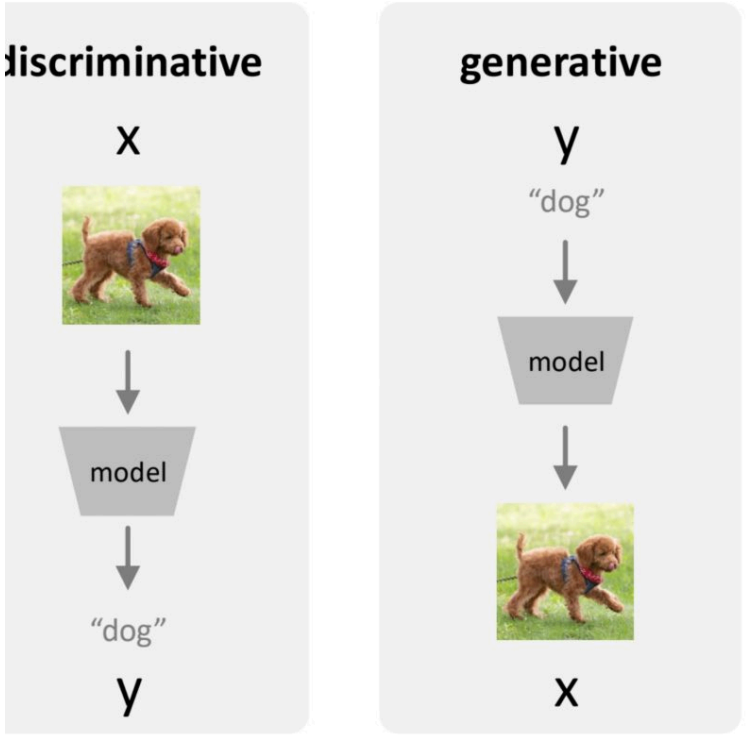
One condition can admit many valid outputs



Several outputs can satisfy the same condition

Discriminative and Generative Tasks

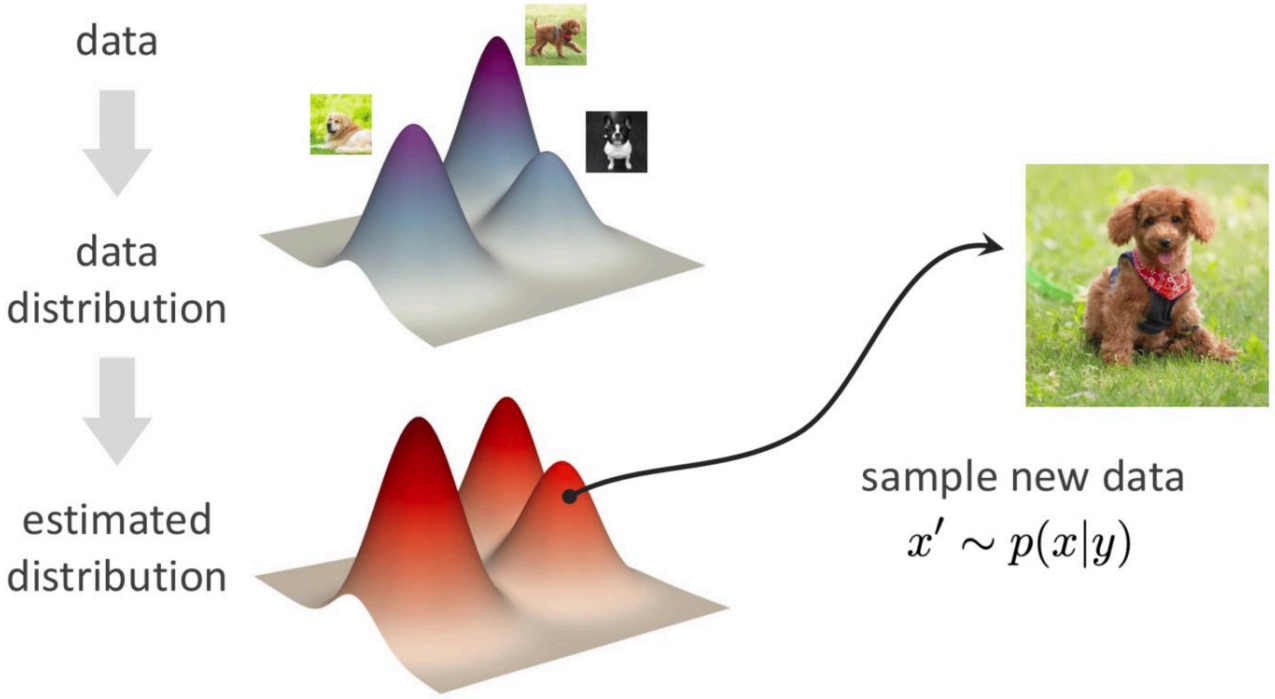
Prediction reverses direction when data become the output



The generated object is the data itself

Data and Model Distributions

Sampling uses an approximation learned from observed data

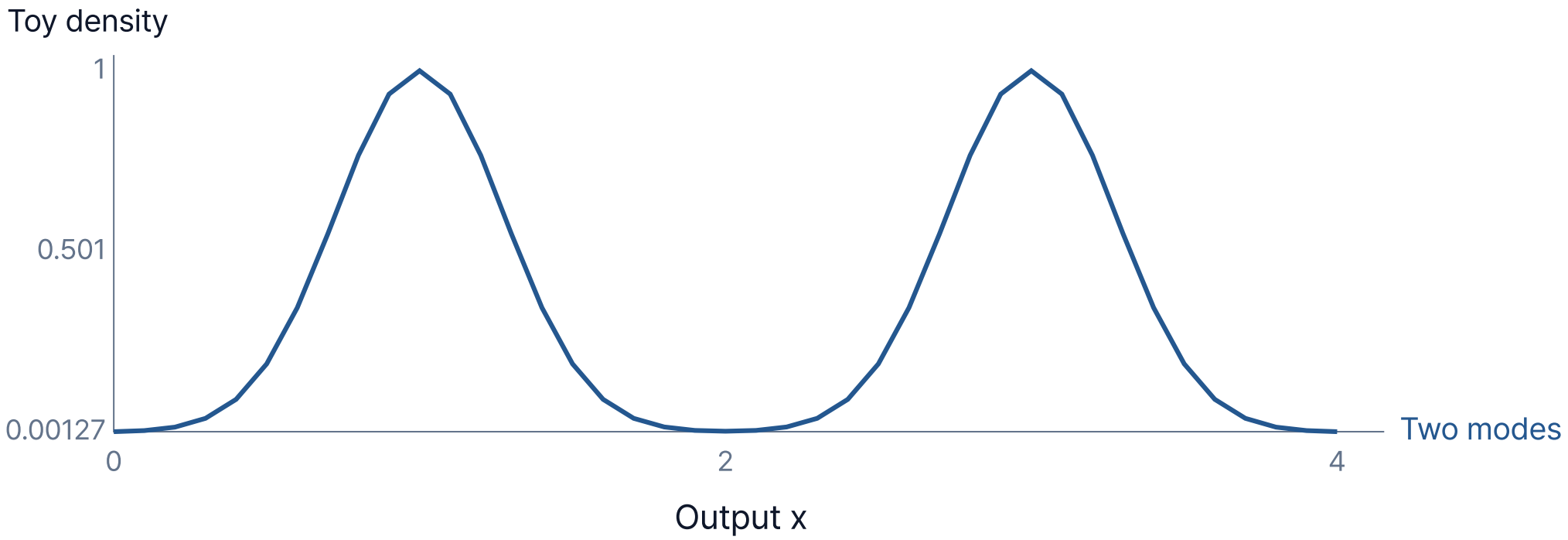


A learned density approximates observed variation

https://arxiv.org/abs/2006.11239

Multimodal Targets

An average can fall between valid modes



The mean can lie between plausible modes

Sampling and Randomness

A seed fixes draws under a fixed implementation

```
rng = np.random.default_rng(7)
samples = rng.normal(size=5)
assert samples.shape == (5,)
```

Demo: Image Seeds

SD-Turbo with one prompt and multiple seeds

Code | Results

Run in Colab ↗

```
prompt = "A fluffy white dog as an astronaut on the moon"
images = [generate_t2i(prompt, seed=s, num_inference_steps=2)[0] for s in [0, 1,
2, 3]]
show_image_grid(images, titles=["Seed 0", "Seed 1", "Seed 2", "Seed 3"], cols=4)
```

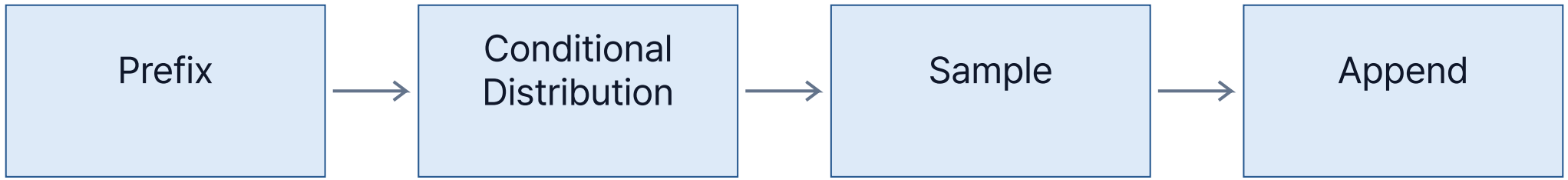
Change the recorded seed. What stays fixed and what changes?

These are actual generated images; the browser displays the measured runs

https://huggingface.co/stabilityai/sd-turbo

Autoregressive Generation

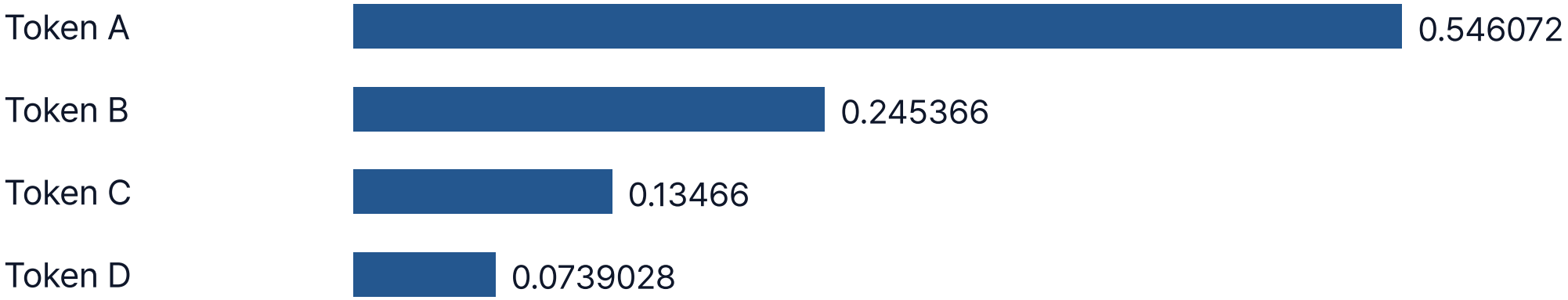
A joint distribution can be sampled through conditionals



A sampled prefix conditions the next draw

Sampling Temperature

Rescaling logits changes probability concentration



Probability at temperature one

Temperature changes how strongly large logits dominate

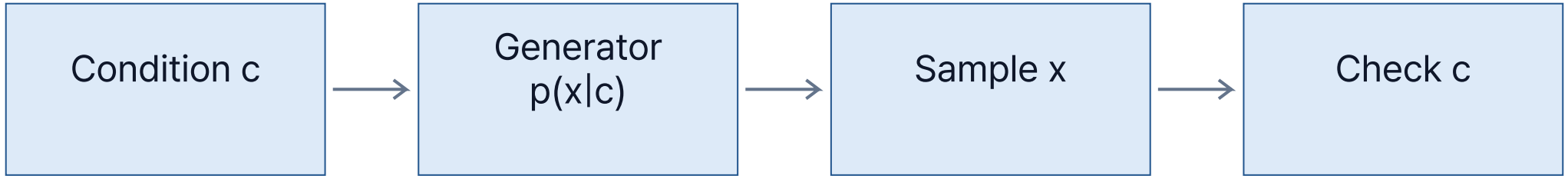
Caveat: Diversity and Correctness

A wider sample distribution can include more invalid outputs

Control	Changes	Does not certify
Temperature	Concentration	Correctness
Seed	Random draw	Quality
More samples	Coverage	Best sample validity

Conditional Generation

A condition changes the distribution being sampled



The generated sample must still satisfy its condition

02 · Denoising Models

01 Sampling Distributions

From one answer to many

02 **Denoising Models**

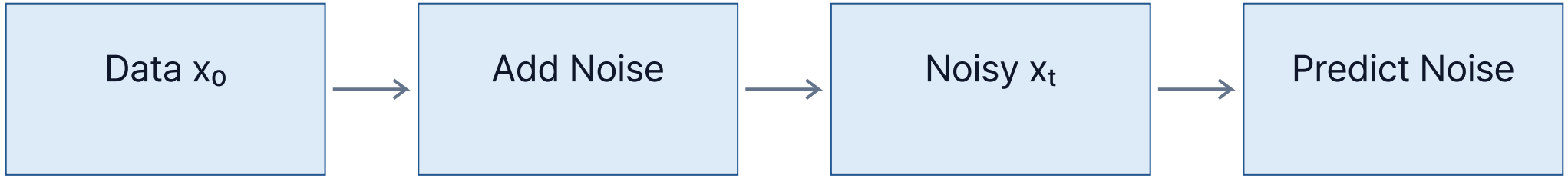
Learn a recovery field

03 Flow and Editing

Control a generation path

Why Add Noise

A controlled corruption process creates a supervised recovery task



Known corruption supplies a training target

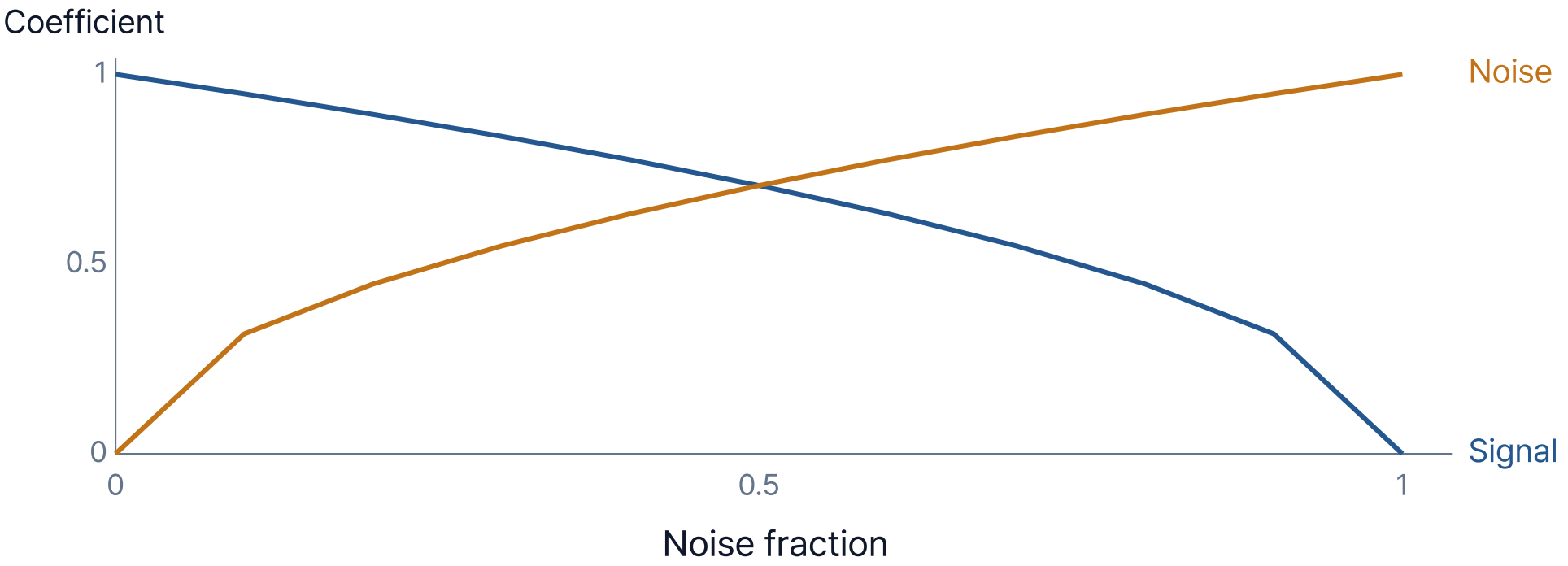
Forward Noising

Signal strength shrinks while random noise grows

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1 - \bar{\alpha}_t)}\, \varepsilon$$

Signal and Noise Weights

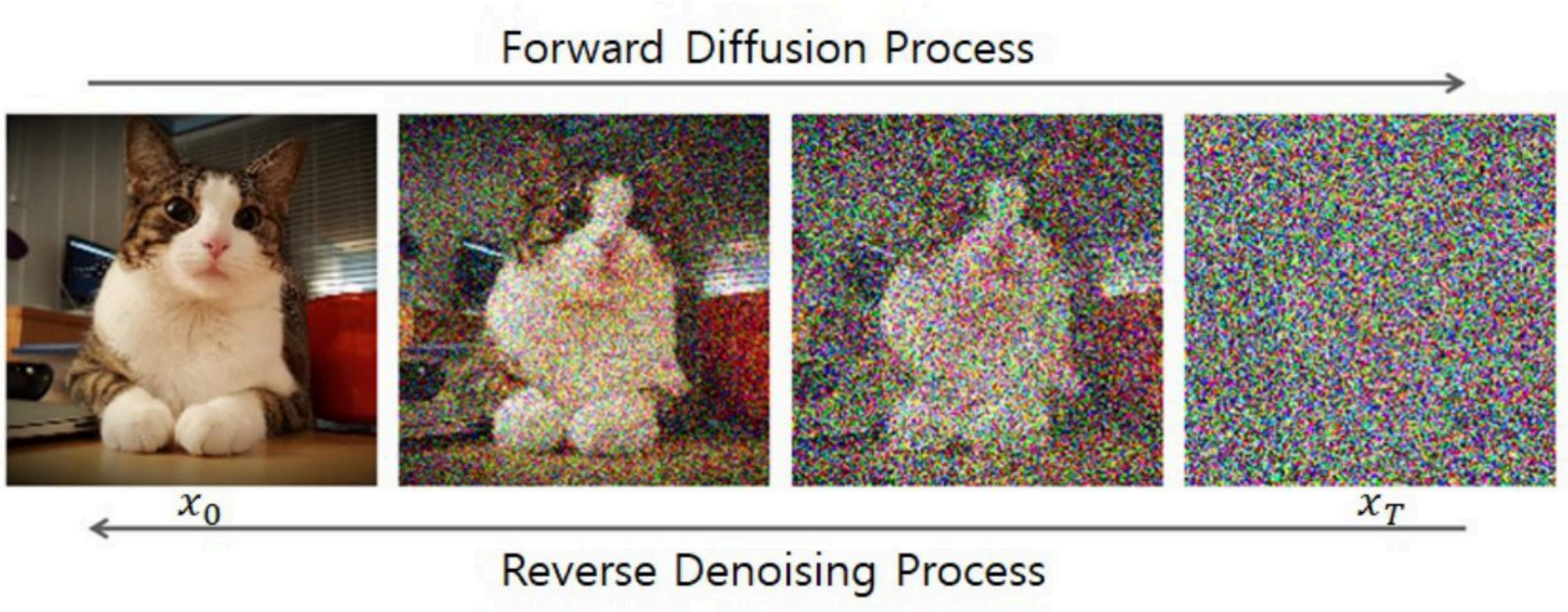
The schedule changes the mixture over time



The signal contribution fades as noise increases

Denoising Diffusion

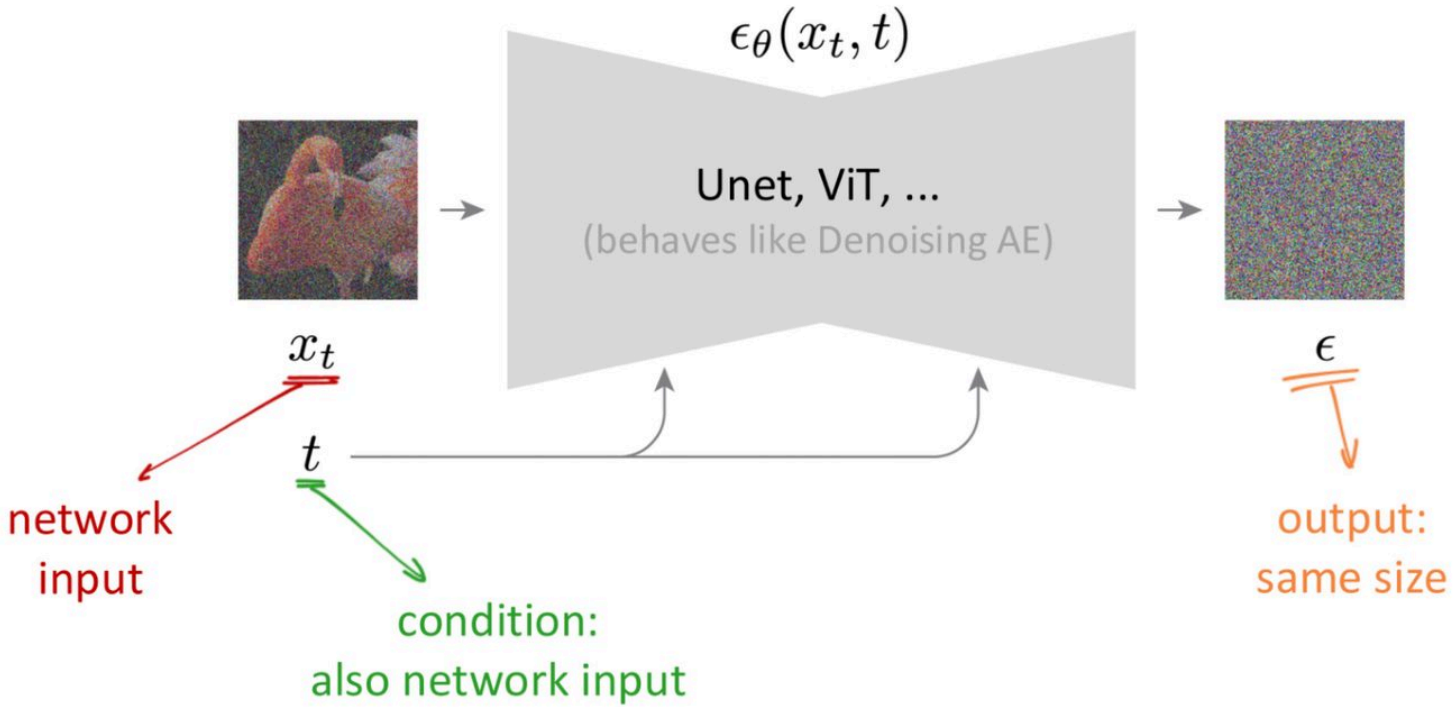
Generation reverses a gradual corruption process



The reverse path gradually recovers structure

Noise Prediction Network

A noisy state and timestep enter the denoiser



The timestep tells the denoiser the corruption level

Noise Prediction Loss

The observed noise provides a regression target

$$L = E[||\varepsilon - \varepsilon_\theta(x_t, t, c)||^2]$$

A Noising Calculation

The same clean value can yield different noisy observations

	x_0	signal	noise	ε	x_t
Example	2	0.6	0.8	0.5	1.6

Here $x_t = 0.6 \times 2 + 0.8 \times 0.5$

Worked Example: Noising and Inversion

Trace the inputs before execution

$x_0=2$; noise $\varepsilon=.5$; signal coefficient $=.6$
noise coefficient $=\sqrt{(1-.6^2)}=.8$; $x_t=1.6$
Predicted $\varepsilon=.5+.1=.6$; $\hat{x}_0=(x_t-.8\times.6)/.6$.

Demo: Noising and Inversion

Executable educational calculation; no pretrained inference

Reduce the signal coefficient; compare reconstruction error under the same prediction bias

Signal coefficient 0.6 Noise prediction bias 0.1

x_0=2; ε=.5; x_t = signal × x_0 + noise × ε

noise=0.800; x_t=1.600; predicted ε=0.600

recovered x_0 = 1.867; error = -0.133

Scalar noising and inversion with an explicitly supplied prediction error; no learned denoiser.
Questions: Does lower signal amplify error? Can x_t determine x_0 without assumptions?

Answer: Noising and Inversion

Same inputs as the initial demo state

$x_t = 1.6$
recovered $x_0 = 1.866667$
error $= -.133333$

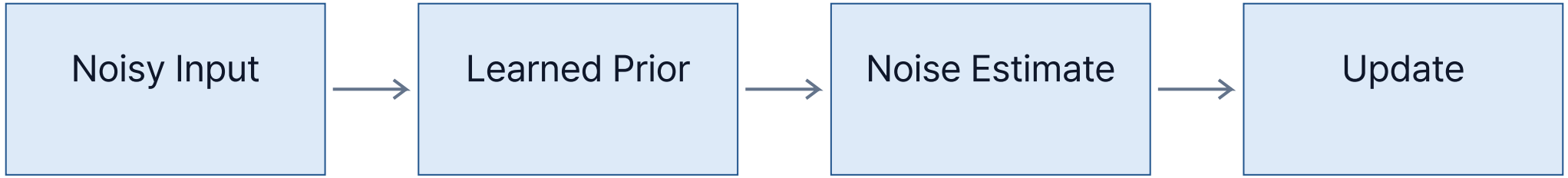
Failure: Noising and Inversion

Identify the assumption that breaks

```
The prediction error is supplied, not learned
recovery with oracle noise does not demonstrate a trained diffusion model
```

Denoising Is Conditional

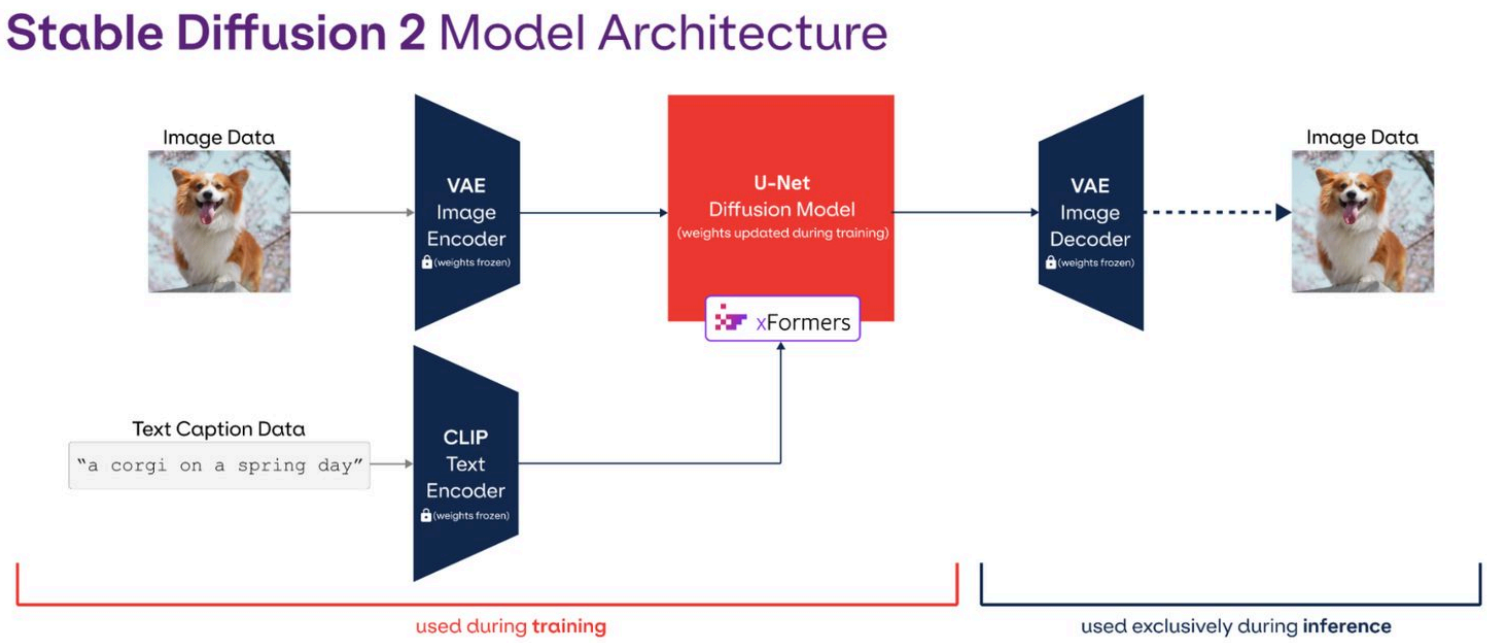
Noise recovery depends on the learned data prior



A learned prior guides the recovery step

Latent Diffusion

A compact representation reduces the generation workspace

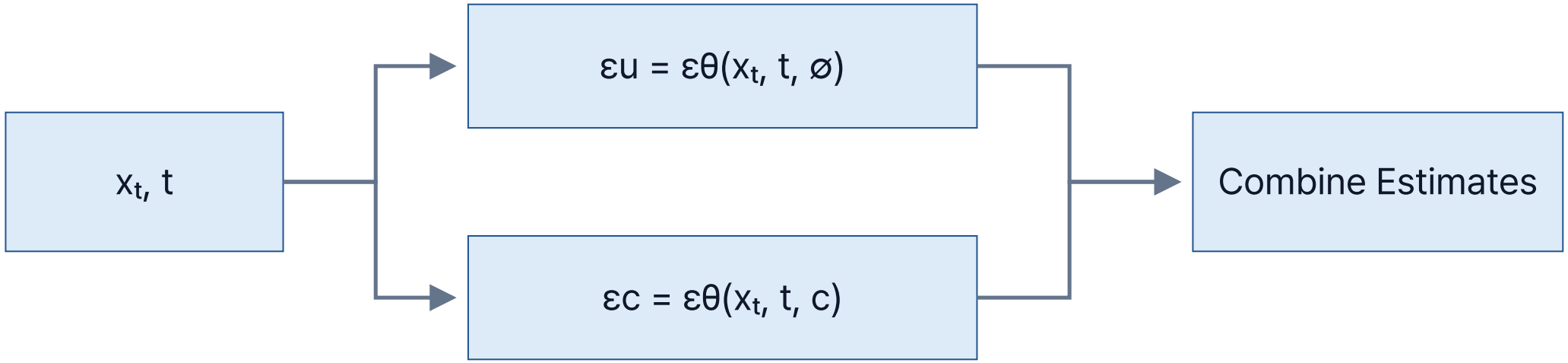


The denoiser works in a compressed image representation

https://arxiv.org/abs/2112.10752

Conditional Guidance

Guidance combines conditional and unconditional predictions



$$\varepsilon guided = \varepsilon u + s\,(\varepsilon c - \varepsilon u)$$

A stronger guidance scale changes the sampling direction

Demo: Prompted Image Generation

Actual SD-Turbo prompt experiments

Code | Results

Run in Colab ↗

```
prompt = "A small cat wearing a tiny backpack, watercolor illustration"
image, seconds = generate_t2i(prompt, seed=7, num_inference_steps=2)
plt.imshow(image); plt.axis("off"); plt.show()
print(f"Measured time: {seconds:.1f}s")
```

Compare style instructions while keeping the seed fixed. Which detail was lost?

SD-Turbo uses guidance_scale = 0 for these runs

https://huggingface.co/stabilityai/sd-turbo

Caveat: Recovery Ambiguity

Severe corruption can hide multiple possible clean inputs

Clean A	1	0	1
Clean B	0	1	0
Noisy observation	0.5	0.5	0.5

A plausible reconstruction need not be the original

03 · Flow and Editing

01 Sampling Distributions

From one answer to many

02 Denoising Models

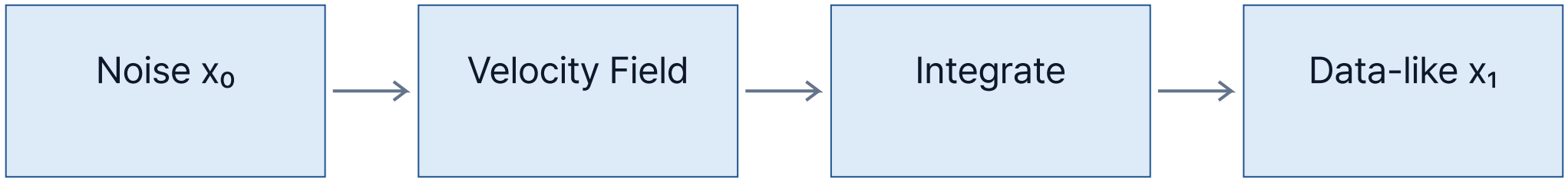
Learn a recovery field

03 **Flow and Editing**

Control a generation path

Why Predict a Flow

A velocity field can transport noise toward data



A learned velocity defines a transport path

Linear Interpolation Path

A simple path joins a noise sample and data sample

$$x_t = (1 - t)x_{noise} + tx_{data}$$

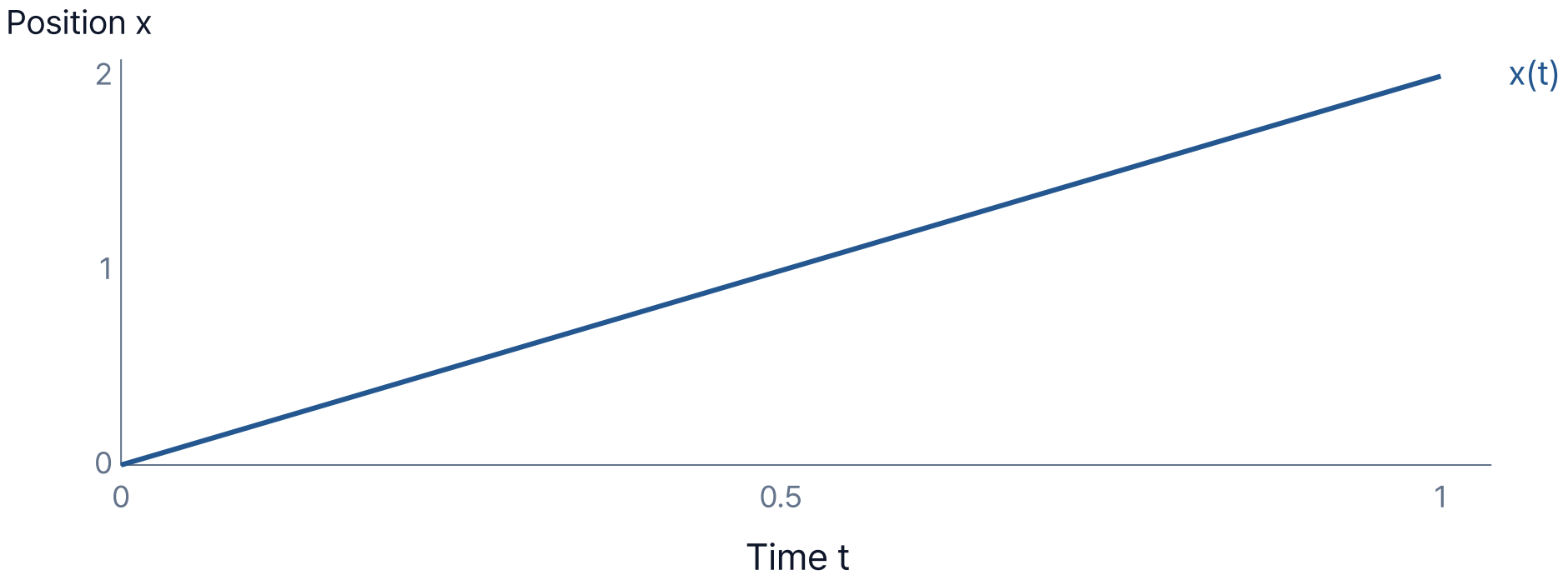
Velocity Target

The linear path has a constant target velocity

```
v_target = x_data - x_noise
```

Euler Integration

A numerical solver turns velocity into successive positions



Position advances by step size times velocity

Worked Example: Velocity Integration

Trace the inputs before execution

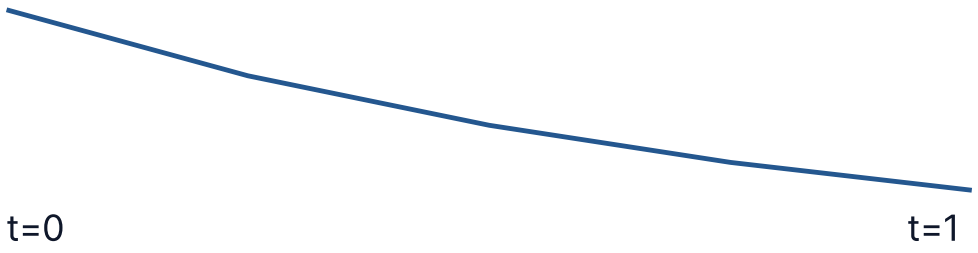
```
Scalar field dx/dt=−x; x(0)=1; integrate to t=1
Euler with 4 steps: x ← x + .25(−x)
Exact solution x(t)=exp(−t).
```

Demo: Velocity Integration

Executable educational calculation; no pretrained inference

Compare 1 and 20 Euler steps; switch the velocity field and explain the error

Euler steps 4 Initial state 1 Velocity v(x,t)=−x



Euler x(1)=0.316

Exact x(1)=0.368

Absolute error=0.051

Update x ← x + (1/steps) v(x,t). Educational scalar ODE, not image generation.
Questions: When is one step exact? Why can fewer steps add numerical error?

Answer: Velocity Integration

Same inputs as the initial demo state

```
Euler path [1,.75,.5625,.421875,.31640625]
exact .367879
absolute error .051473
```

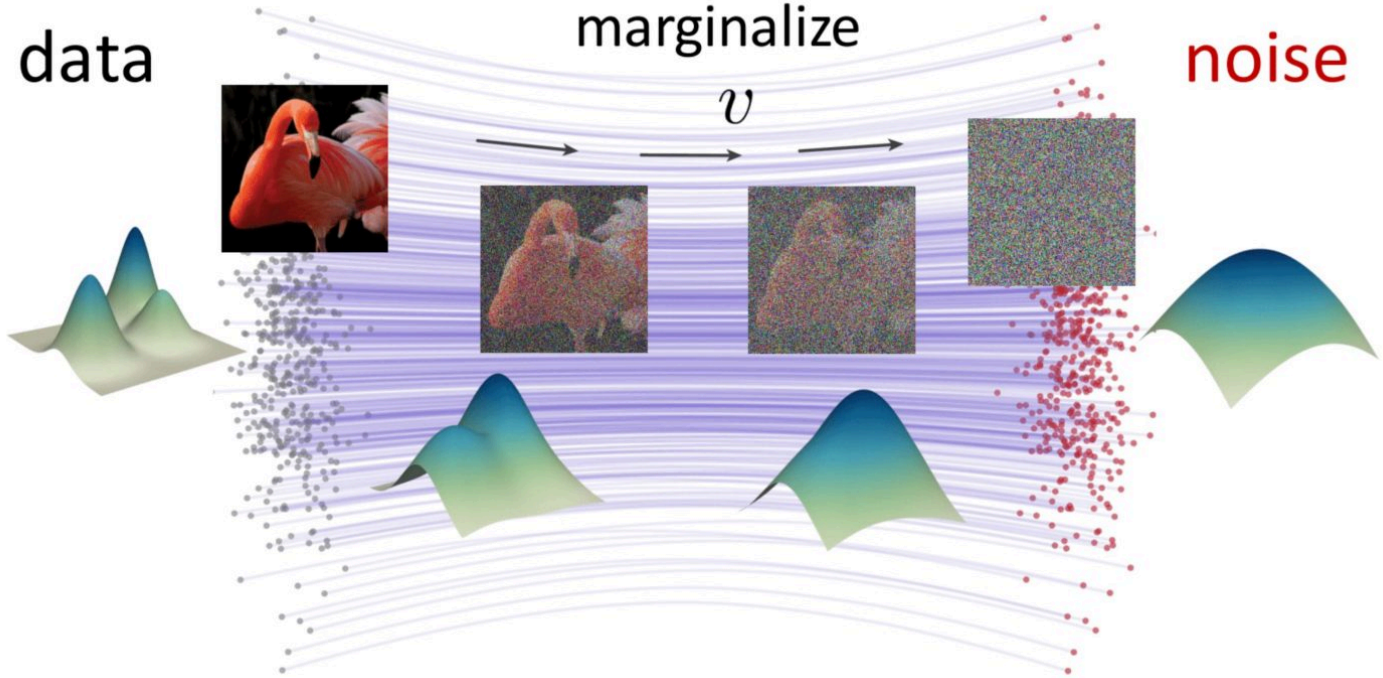
Failure: Velocity Integration

Identify the assumption that breaks

```
A scalar ODE is not an image generator
reducing Euler error does not establish perceptual sample quality
```

Flow Matching Illustration

The learned vector field connects distributions



A velocity field moves a distribution over time

https://arxiv.org/abs/2210.02747

Distillation Tradeoff

A shorter generation path shifts work into training



Illustrative model evaluations

Fewer evaluation steps do not remove the quality check

Demo: Image Generation Steps

Same prompt and seed, different inference steps

Code | Results

Run in Colab ↗

```
prompt = "A corgi wearing a red scarf in a snowy village"
for steps in [1, 2, 4]:
    image, seconds = generate_t2i(prompt, seed=123, num_inference_steps=steps)
    plt.imshow(image); plt.title(f"{steps} steps, {seconds:.1f}s")
    plt.axis("off"); plt.show()
```

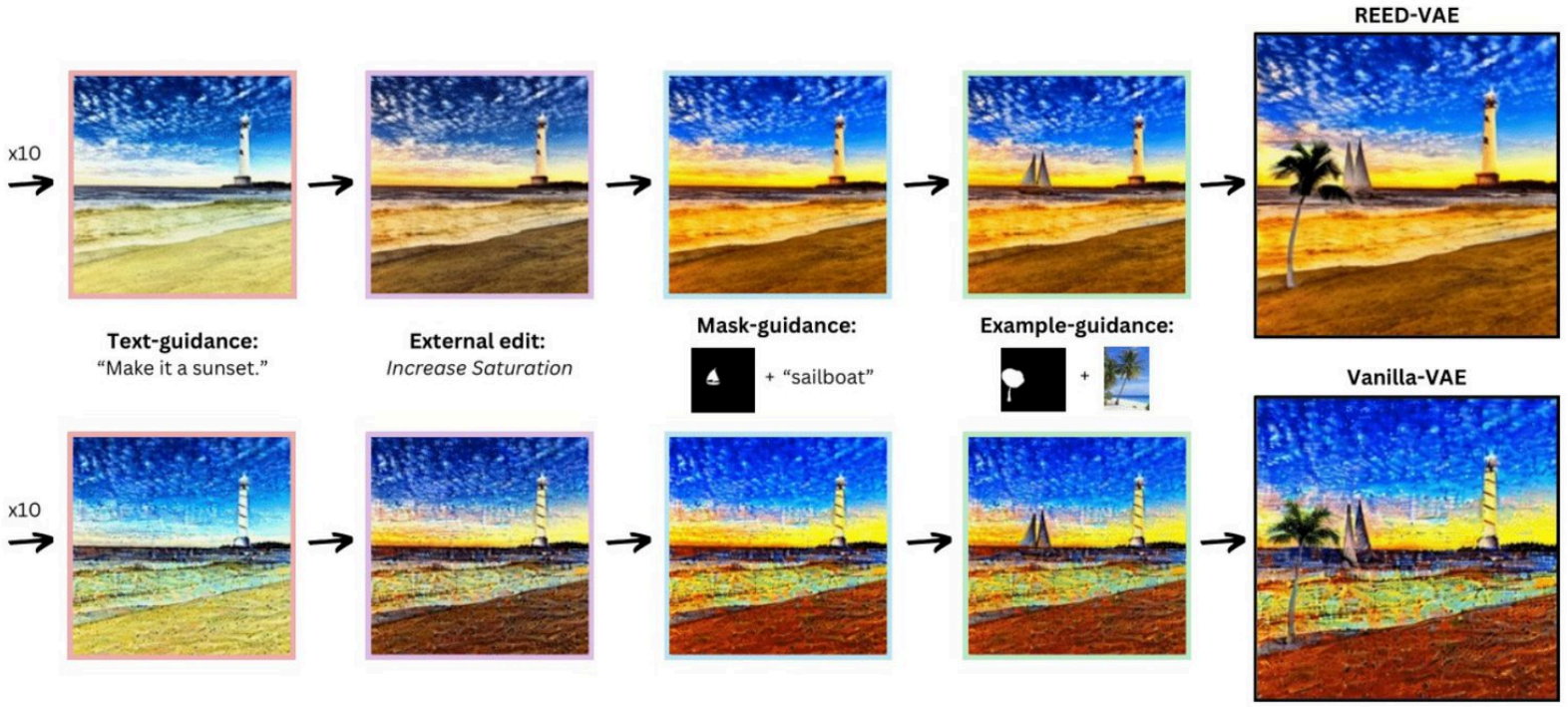
Compare image detail and measured runtime. Do more steps always improve the result?

The timings describe this execution environment

https://huggingface.co/stabilityai/sd-turbo

Iterative Image Editing

A previous image can condition a sequence of edits



Each edit must preserve the unconstrained content

Unified Multimodal Generation

A shared model can couple understanding and image production

Capability	Input	Output
Understanding	Image + question	Text
Generation	Text or image context	Image
Editing	Image + instruction	Revised image

Demo: Show-o2 Understanding and Generation

Show-o2 · Understanding and generation

Code | Results

Run in Colab ↗

```
question = "Describe this pet and its visible background in one sentence."
briefing = run_showo2_mmu(pet_image_path, question)
print(briefing)
# The next experiment uses the actual model response as its image prompt.
```

Follow the generated prompt into the image stage. Which input details survive?

The chain must use the actual model response, not a manual substitute

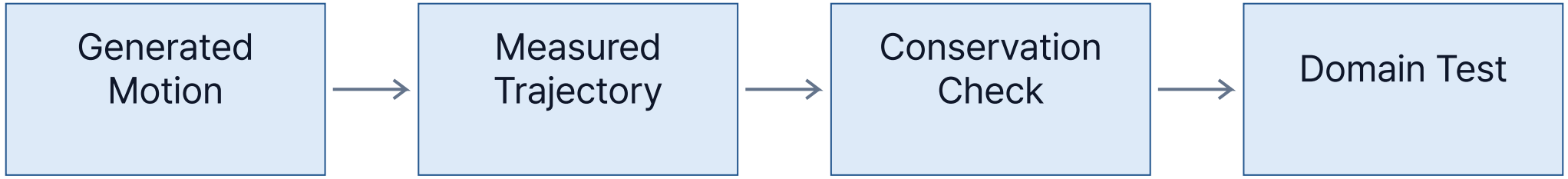
Generative Evaluation

Assess validity, diversity and data resemblance separately

Question	Test
Condition satisfied?	Targeted checks
Samples vary?	Repeated seeds
Data-like outputs?	Held-out comparison

Caveat: Simulation Claims

Plausible video alone does not establish physical accuracy



Physical claims require measurements beyond appearance

Takeaway: Generative Models

01 A generated sample is drawn from a learned approximation

02 Training targets and sampling rules serve different roles

03 Condition fidelity and physical validity need explicit checks